Service Charter Document

Fest Management Platform

Purpose

To automate and centralize the lifecycle of college hackathons and technical fests, replacing fragmented manual tools with a single, high-performance platform for MITAOE.

Objective

To make the process of organizing, registering for, and managing hackathons:

- **Time Saving:** Reducing the administrative workload for club coordinators by automating team formation and registration tracking.
- **Precise and Accurate Result:** Ensuring zero data duplication and accurate payment verification for paid events.
- **Well Structured and Managed:** Providing a clear hierarchy for college admins to oversee multiple clubs and fests from one dashboard.

Demand / Opportunity

There is a significant opportunity as, currently, no dedicated, all-in-one hackathon platform is tailored for the MITAOE ecosystem. Most developers and club leads are forced to use a mix of Google Forms, WhatsApp, and Excel, which fails when handling long and complex participant models and team-based relationships.

Business Requirements

- **BR1: User Authentication**

  Description:
  A secure authentication mechanism must be implemented to control access to the platform.

  Requirement:
  Users including **Admins, Coordinators, and Students** must log in using a valid **username/email and password** to access the platform. The system must verify credentials before granting access.

  Justification:
  This ensures **secure access control**, protects sensitive student data, and prevents unauthorized users from modifying event or platform settings.

- **BR2: Event / Hackathon Creation**

  Description:
  The system must allow organizers to create and configure events or hackathons easily.

  Requirement:
  Authorized users should be able to create an event by defining key attributes such as:

  - Event Name
  - Date and Time
  - Venue
  - Team Size
  - Eligibility Criteria
  - Event Category

  Justification:
  This functionality enables organizers to quickly deploy different types of events such as **technical fests, cultural programs, workshops, or hackathons**.

- **BR3: Relationship Mapping (Team Management)**

  Description:
  The system must support structured relationships between participants and teams.

  Requirement:
  The platform must allow mapping relationships between **Students, Teams, Mentors, and Projects**. Multiple students should be able to join a team.

  Justification:
  This automates complex **team formation and project allocation**, which is one of the most challenging aspects of managing hackathons and collaborative competitions.

- **BR4: Event Reusability**

  Description:
  The platform must allow organizers to reuse previous event configurations.

  Requirement:
  Existing event templates, registration structures, and configurations should be reusable for future events with necessary modifications.

  Justification:
  This reduces repetitive setup work and allows clubs to **replicate successful event structures efficiently** for future semesters.

Technical Requirements

To manage high-traffic college fests and complex nested hierarchies, the following technical standards are established:

- **System Scalability:**
  The architecture must support horizontal scaling to handle up to **500 concurrent users** during peak registration windows for major fests.

- **Multi-Tier Nested Hierarchy:**
  Core logic must support the following relational structures:

  - College → Fest → Club → Event

  - College → Club → Standalone Event

- **Access Control (RBAC):**
  Implementation of **Role-Based Access Control** for four distinct tiers:

  - Super Admin (University)

  - Admin (College)

  - Moderator (Club Leads)

  - User (Students)

- **Transaction Integrity:**
  Atomic transaction handling must be implemented to prevent **over-booking of limited-capacity workshops or competitions**.

- **Notification Engine:**
  Real-time delivery of schedule changes and registration confirmations through **asynchronous messaging and automated SMTP triggers**

Technological Requirements

We have evaluated various stacks to ensure long-term stability and performance.

Component	Selected Technology	Primary Reason
Frontend	React	Provides a component-based architecture that allows for highly interactive and reusable UI elements for fests.
Backend	Java with Spring Boot	Offers enterprise-grade security and superior multi-threading capabilities for handling thousands of simultaneous registrations.
Database	PostgreSQL	Ensures strict data integrity for the complex relationships between colleges, clubs, and events.

Frontend Framework Comparison

Feature	React (Chosen)	Angular	Vue.js
Learning Curve	Moderate	High	Low
Performance	High (Virtual DOM)	High (Real DOM)	High (Virtual DOM)
Ecosystem	Massive (Tailwind, Material UI)	Full-featured but rigid	Smaller than React
Decision	**React**	Not Selected	Not Selected

Database Postgres: Why PostgreSQL?

A rigorous comparison was conducted between SQL and NoSQL variants to determine the best fit for Unbound's relational data.

Feature	PostgreSQL (Selected)	MySQL	MongoDB
Data Model	Relational (Strict)	Relational (Strict)	Document (Flexible)
Joins	Excellent (Nested Data)	Good	Poor (Requires Lookups)
ACID Compliance	Full (High Integrity)	Full	Document-level only
Scalability	Vertical and Horizontal	Mostly Vertical	Excellent Horizontal
Nested Logic	JSONB support for flexible fields	Limited JSON support	Native JSON

PostgreSQL is the chosen database because the "Fest > Club > Event" relationship is inherently relational. PostgreSQL provides the strict data integrity needed for registrations while offering JSONB for any unstructured event metadata.

Resources Needed

- **Infrastructure:** Cloud Compute(2-4 VCPUs ,linux based os with 8gb ram and 20gb disk space) for Java App, RDS for Postgres, Cloud Storage for assets, Domain with SSL (Cloudflare), and SendGrid for transactional emails.
- **Development Tools:** GitHub for version control, Jira for Agile sprint management, Maven for Java dependency management, and Postman for API documentation.
- **Security Tools:** JWT for session tokens, Spring Security for authentication, and Bcrypt for password hashing.

Human Resources

1. **1 Project Lead (Scrum Master):** Responsible for sprint planning, removing blockers, and stakeholder alignment.
2. **2 Frontend Developers:** Responsible for the Student Discovery Portal and the Club Organizer Dashboard using React.
3. **2 Backend Developers:** Responsible for Java API architecture, Database Schema design, and Security protocols.
4. **1 UI/UX Designer:** Responsible for mobile-first design, prototyping, and accessibility compliance.
5. **1 QA Engineer:** Responsible for automated unit testing (JUnit) and End-to-End (E2E) registration flow testing.

PESTEL Analysis:

Factor	Impact on Project	Example
Political	No direct political restrictions affecting the platform	MITAOE operates independently for internal event management
Economic	Reduces manual management cost for clubs	Less dependency on paid event tools
Social	Encourages student participation and engagement	Students easily discover events
Technological	-	-
Environmental	Digital platform reduces paperwork and manual forms	Eco-friendly event registration
Legal	Must comply with data protection and payment regulations	Secure handling of student data and payment transactions

Risk Analysis:

Risk	Description	Probability	Impact	Mitigation Strategy
Software Compatibility	Framework or dependency updates may break features	Medium	Medium	Regular dependency updates and testing
Database Server Downtime	Failure of DB server can stop event registration	Medium	High	Use backup replica and failover
Cloud Server Failure	Cloud Services outage may cause service disruption	Low	High	Multi-region deployment

Budget Overrun	Development cost may exceed estimate	Medium	Medium	Strict budget monitoring
Security Breach	Unauthorized access to student data	Low	High	Use **bcrypt password hashing**, JWT authentication , and role-based access control
Scalability Issues	High traffic during registrations	Medium	High	Auto-scaling cloud infrastructure

Timeline / Milestone:

Phase	Milestone	Tasks	Timeline
1	Requirement Analysis & Planning	- Gather and Document project requirements - Define & construct functional flow of the application - Identify third-party services and dependencies - Design UI mockups and wireframes	Week 1
2	Database Design and API Development	- Design database schema - Set up database configurations - Develop API endpoints - Implement user authentication setup - Perform API testing using postman	Week 2
3	UI Development and API Integration	- Develop UI components and pages - Implement state management - Connect frontend with backend - Implement error handling & validation - Optimize API calls	Week 3
4	Testing & Debugging	- Perform UI/UX testing - Perform API integration testing - Fix bugs and optimize performance	Week 4
5	Deployment & Final Review	- Set up cloud hosting (AWS) - Implement CI/CD pipelines - Monitor and resolve deployment issues - Conduct final review and documentation	Week 4

Stakeholder Table :

Stakeholder	Role
MITAOE Admin	Oversees events and clubs
Club Coordinators	Create and manage events
Students	Register and participate in events
Development Team	Build and maintain the platform
QA Team	Ensure system quality